Universidad Andrés Bello
Conectar · Innovar · Liderar

Optimizacion

0-1 Integer Programming

Benjamin Gallegos, Felipe Levi, Felipe Sanhueza

Problema

El Problema de Programación Entera 0-1 (o 0-1 Integer Programming) consiste en seleccionar el mejor subconjunto de opciones indivisibles de un grupo finito para maximizar beneficios o minimizar costos.

Ejemplo Ilustrativo: Asignación Estratégica de Capital.

Una empresa tiene como presupuesto máximo de $14.000 para invertir en cuatros posibles proyectos indivisibles.

- Proyecto 1: Costo $5.000 | Beneficio (VPN) $8.000
- Proyecto 2: Costo $7.000 | Beneficio (VPN) $11.000
- Proyecto 3: Costo $4.000 | Beneficio (VPN) $6.000
- Proyecto 4: Costo $3.000 | Beneficio (VPN) $4.000

Para modelar esto, definimos primero la variable xj {0,1}, donde xj=1 si se invierte en el Proyecto j, y 0 en caso contrario.

- El objetivo es maximizar el VPN total, considerando la restricción de no superar las $14.000.

Modelado Matemático

COMPONENTES Y VARIABLES

CONJUNTOS

N: Decisiones (n)
M: Restricciones (m)

VARIABLES

$$x_j \in \{0, 1\}$$

PARÁMETROS

c_j	Costo/Beneficio impacto.
a_{ij}	Coeficiente tecnológico.
b_i	Límite de recursos.

- Las variables actúan como interruptores lógicos (1=Selección, 0=Rechazo).

COMPLEJIDAD COMPUTACIONAL

Versión de Optimización

- Busca el valor de decisión exacto que maximice/minimice el beneficio absoluto en el espacio de soluciones.

Versión de Decisión

- Transforma el problema en una pregunta logística (Sí/No) mediante un umbral **K**.

$$c^T x \geq K$$

FORMULACIÓN ALGEBRAICA

FUNCIÓN OBJETIVO (OPTIMIZACIÓN GLOBAL)

$$\max \sum_{j=1}^{n} c_j x_j$$

RESTRICCIONES DE CAPACIDAD BASE

$$\sum_{j=1}^{n} a_{ij} x_j \leq b_i \quad \forall i \in M$$

MODELADO DE LÓGICA AVANZADA

Condicional

$$x_a \leq x_b$$

A depende de B

Excluyente

$$\sum x_k \leq 1$$

Máximo 1 opción

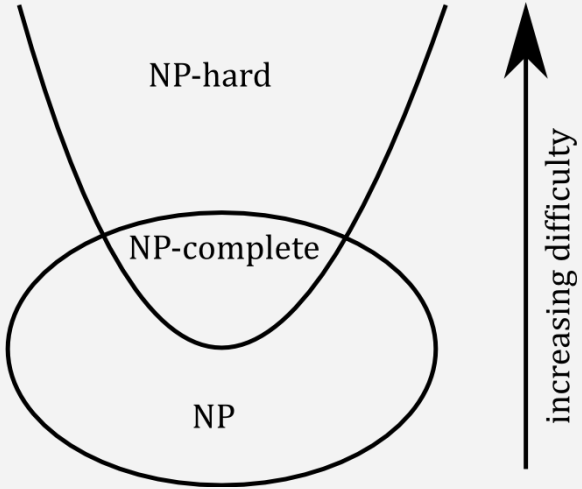
Big M

$$y_j \leq M x_j$$

Binaria vs Continua

Complejidad

Para demostrar que el problema pertenece a la clase de NP-Completo, este debe ser perteneciente a tanto la clase NP como a la de NP-HARD



¿POR QUÉ ESTÁ EN NP?

Un problema es NP si podemos verificar una solución en tiempo polinomial. Para un vector propuesto, el proceso requiere solo tres validaciones simples:

- **Tipado Binario:** Validar $x \in \{0,1\}^n$ toma $O(n)$.
- **Restricciones:** Verificar $Ax \leq b$ toma $O(m \times n)$.
- **Umbral:** Calcular $c^T x \geq K$ toma $O(n)$.

COSTO DE VERIFICACIÓN

$$C(x) = O(m \times n)$$

INTUICIÓN NP-HARD

Se demuestra mediante la Reducción Polinomial de Karp. Cualquier instancia de 3-SAT puede transformarse en un modelo 0-1 IP.

Transducción (Ejemplo):

Cláusula: $u_1 \vee \neg u_4 \vee \neg u_7$

$$x_1 + (1 - x_4) + (1 - x_7) \geq 1$$

Formato Canonico:

$$-x_1 + x_4 + x_7 \leq 1$$

Implementacion

Python

Enfoque Base

- Recursión pura (backtracking) mediante la clase IPBase. Optimización de memoria pasando estados directamente en la pila de llamadas.

Mejora Implementada

- Pre-ordenamiento por densidad (heurística) y poda drástica mediante relajación fraccional en el método cota.

Decisiones Relevantes

- Herencia de referencias para backtracking directo, uso de lambdas para ordenamiento eficiente.

C++

Enfoque Base

- Clase SolverIPBase con inicialización de vectores constantes para eludir copias de memoria redundantes en la recursión.

Mejora Implementada

- Topología compacta con struct Elemento y comparador personalizado en std::sort. Conteo asintótico de nodos.

Decisiones Relevantes

- Control nativo del hardware y monitoreo estricto para contrastar el esfuerzo computacional podado.

Java

Enfoque Base

- Uso de arreglos primitivos globales para prevenir la sobresaturación del Garbage Collector ante la recursión masiva.

Mejora Implementada

- Clase interna estática Item con interfaz Comparable. Poda temprana validando cota() contra el mejor beneficio global.

Decisiones Relevantes

- Diseño orientado a objetos type-safe para evitar latencias estructurales de la JVM en concurrencia.

Resultados experimentales

N Evaluado	Ramas Teóricas Aprox. (2N)	Nodos Base (C++)	Nodos Base (Java)	Nodos Mejorada (C++)	Nodos Mejorada (Java)	Eficiencia de Poda
10	1,024	1,176	1,154	-	-	-
15	32,768	39,212	39,19	-	-	-
20	1,048,576	-	1,220,917	93	239	> 99.99%
25	33,554,432	38,085,072	38,253,667	-	-	-
30	1,073,741,824	1,162,177,087	1,204,984,676	-	-	-
40	$\approx 1.09 \times 10^{12}$	Inabordable	Inabordable	134	337	> 99.99%
60	$\approx 1.15 \times 10^{18}$	Inabordable	Inabordable	388	147	> 99.99%
100	$\approx 1.26 \times 10^{30}$	Inabordable	Inabordable	590	1,614	> 99.99%
1000	$\approx 1.07 \times 10^{301}$	Inabordable	Inabordable	16,108	8,31	> 99.99%

- Para evaluar el rendimiento real de los algoritmos y caracterizar su comportamiento predictivo, se diseñó un cuasi-experimento confrontando las implementaciones con instancias de volúmenes de coeficientes crecientes

Resultados experimentales

Lenguaje	Versión	N=10	N=15	N=20	N=25	N=30	N=40	N=60	N=100	N=1000
Python	Base	0.000143	0.004425	0.135019	4.556.120	140.394.441	Inabordable	Inabordable	Inabordable	Inabordable
Python	Mejorada	-	-	0.000045	-	-	0.000093	0.000251	0.000857	-
Java	Base	0.000163	0.000595	0.007195	0.109581	3.385.958	Inabordable	Inabordable	Inabordable	Inabordable
Java	Mejorada	-	-	0.000088	-	-	0.000198	0.000041	0.000119	0.003636
C++	Base	0.000010	0.000404	-	0.293404	10.733.893	Inabordable	Inabordable	Inabordable	Inabordable
C++	Mejorada	-	-	0.000005	-	-	0.000013	0.000026	0.000045	0.003313

Conclusiones

Aprendizaje

- Frente a los límites de la clase NP, la solución no reside principalmente en la fuerza bruta, sino en la inteligencia algorítmica. La capacidad de proyectar cotas matemáticas y podar espacios de búsqueda inútiles es lo que permite resolver de forma eficiente instancias de 0-1 IP en tiempos viables.

Limitaciones y Mejoras

- **Limitación**: El peor caso sigue siendo intratable debido a la clase NP.
- **Mejoras**: Implementar técnicas de Branch-and-Cut o explorar metaheurísticas para aproximaciones rápidas.

Universidad Andrés Bello
Conectar · Innovar · Liderar

Optimizacion

0-1 Integer Programming

Benjamin Gallegos, Felipe Levi, Felipe Sanhueza